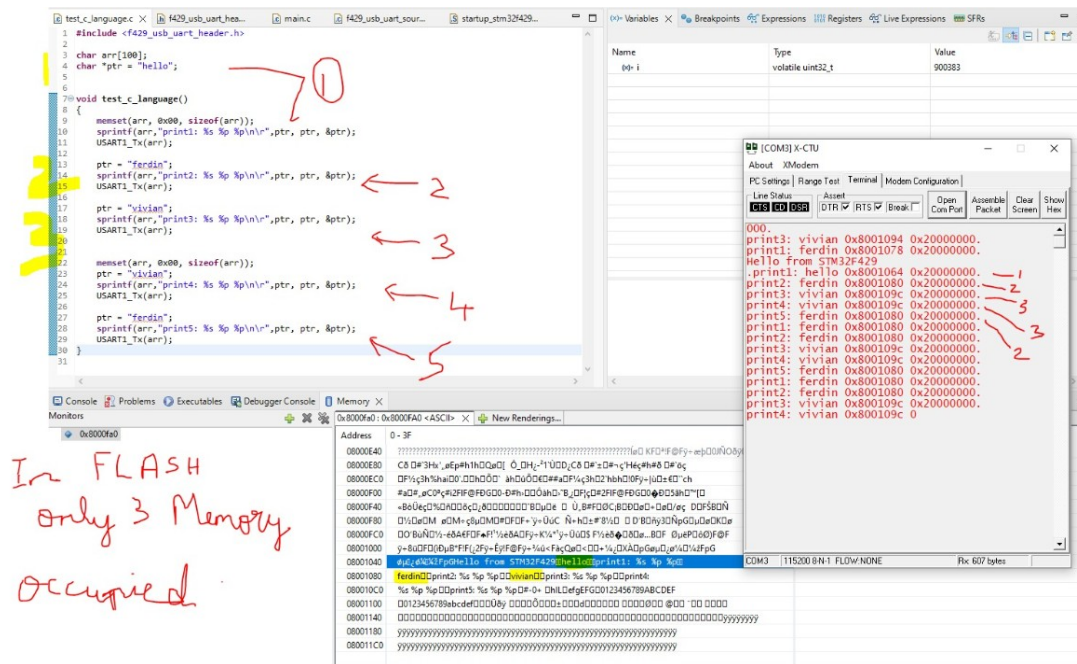




## #include "header.h" vs #include <header.h>

### 1. #include "header.h"

- Meaning: "Look in MY folder first"
- Search order:
  - Current project folder
  - Then compiler/system folders

### 2. #include <header.h>

- Meaning: "Look in SYSTEM folders only"
- Search order:
  - Compiler/Library folders only
  - Use for: Standard library + MCU vendor files

## CODE SUMMARY

char arr[100]; // Array in RAM

char \*ptr = "hello"; // Pointer to String Literal in FLASH

void test\_c\_language()

{

    memset(arr, 0x80, sizeof(arr)); // Fill RAM array with 0x80

    ptr = "ferdin"; // Reassign pointer to new literal

    ptr = "vivian"; // Reassign pointer again

}

## KEY CONCEPTS - BULLET POINTS

### String Literal Storage

- "hello", "ferdin", "vivian" are stored in FLASH memory at compile time
- Only 3 unique literals = Only 3 memory locations in FLASH. Hence your note: "In FLASH only 3 Memory occupied"

### "Pointer char \*ptr

- ptr itself is in RAM. It just holds an address
- When you do ptr = "ferdin", you are NOT copying string. You are changing the address ptr points to
- Address changes: 0x8001064 -> 0x8001078 -> 0x800109C as seen in UART output

### Array char arr[100]

- arr is 100 bytes allocated in RAM
- &arr is address of array in RAM: 0x20000000
- sizeof(arr) = 100 always

### Why ptr address changes but &ptr stays same?

- %p ptr = Address of data in FLASH that ptr points to. This changes
- %p &ptr = Address of pointer variable itself in RAM. This stays constant: 0x20000000

### memset on arr

- Only affects RAM array. Does NOT affect string literals in FLASH
- FLASH is read-only at runtime

## UART OUTPUT ANALYSIS

- print1: ptr points to "hello" -> 0x8001064
- print2: ptr points to "ferdin" -> 0x8001078
- print3: ptr points to "vivian" -> 0x800109C
- print4: ptr points to "vivian" again -> same address reused
- print5: ptr points to "ferdin" again -> same address reused
- Conclusion: Compiler stores each unique literal only once in FLASH

## LEARNINGS

- Pointers to string literals point to FLASH, not RAM
- Reassigning pointer is cheap. No memory copy happens
- To modify string, use array in RAM, not pointer to literal
- This saves RAM in embedded systems

## IMPORTANT WARNING

- ptr = "hello"; ptr[0] = 'H'; -> CRASH. Because FLASH is read-only
- Use char arr[] = "hello"; if you need to modify

### ### \*\*What image proves\*\*

1. \*\*Flash optimization\*\*: Compiler put only 3 strings in flash even though you assigned 5 times
2. \*\*Pointer vs Array\*\*: `ptr` changes address, `arr` address is fixed at 0x20000000
3. \*\*RAM vs FLASH\*\*: `&ptr` and `arr` are in RAM `0x2000xxxx`. String data is in FLASH `0x8000xxxx`

### Source Code:

```
#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

char arr[100];
char *ptr = "hello";

void test_c_language()
{
    memset(arr, 0x00, sizeof(arr));
    sprintf(arr, "print1: %s %p %p\n\r", ptr, ptr, &ptr);
    USART1_Tx(arr);

    ptr = "ferdin";
    sprintf(arr, "print2: %s %p %p\n\r", ptr, ptr, &ptr);
    USART1_Tx(arr);

    ptr = "vivian";
    sprintf(arr, "print3: %s %p %p\n\r", ptr, ptr, &ptr);
    USART1_Tx(arr);

    memset(arr, 0x00, sizeof(arr));
    ptr = "vivian";
    sprintf(arr, "print4: %s %p %p\n\r", ptr, ptr, &arr);
    USART1_Tx(arr);

    ptr = "ferdi";
    sprintf(arr, "print5: %s %p %p\n\r", ptr, ptr, &ptr);
    USART1_Tx(arr);
}
```